Deterministic Graph Search and Matrix-Based Validation: A Reproducible Framework for SAT-Oriented Research

Charalampos Nadiotis
PELAOS Tech, Cyprus

30 October 2026
Technical report, version 1.0

Abstract

This report presents a reproducible implementation of breadth-first search, iterative depth-first search, bidirectional breadth-first search, Dijkstra's algorithm, and A*. The implementation combines adjacency lists, explicit traversal state, a binary priority queue, and reopening of vertices when a cheaper path is found. Five illustrative graphs yield forty-five exported matrices, including adjacency, incidence, Laplacian, reachability, and all-pairs shortest-path matrices. Exhaustive tests cover all 4,096 directed simple graphs on four vertices; independent weighted tests and algebraic checks complement them. On an open 40×40 grid, Manhattan-guided A* processes 79 vertices while BFS processes 1,600, with both returning a path of cost 78. Obstructed-grid results show a smaller reduction. Two elementary propositions explain why a depth heuristic and exact residual-function merging do not by themselves establish a polynomial-time SAT algorithm. The contribution is a documented experimental and mathematical baseline; the methods are established algorithms, and this report does not establish $\mathsf{P} = \mathsf{NP}$.

Keywords: graph search; A*; BFS; DFS; graph matrices; satisfiability; reproducibility.

1 Purpose and scope

An efficient implementation of graph search is useful both as a computational tool and as a controlled setting for examining proposed complexity arguments. This work develops the restricted, testable milestone described in the author's earlier framework paper [2]. It distinguishes three questions: whether an implementation returns a correct answer, how much work it performs on explicit graphs, and whether the graph representing a succinct input can itself be exponentially large.

The present contribution consists of executable implementations, deterministic experiments, a local matrix collection, explicit mathematical assumptions, and a validation record. It does not introduce BFS, DFS, Dijkstra's algorithm, or A* as new methods. The heuristic-search foundation follows Hart, Nilsson, and Raphael [1]; the finite implementation here also permits zero-cost edges. No claim about the contents or validity of an unavailable earlier proof is needed for these results.

2 Model and implementation contract

Let $G = (V, E)$ be a finite directed graph; undirected input edges are represented by two adjacency entries, except self-loops. Vertices have integer identifiers $0, \ldots, |V| - 1$. An edge has a nonnegative integer cost w. BFS and bidirectional BFS accept only unit-cost graphs. DFS returns a path and its cost, without promising optimality. Weighted shortest paths use Dijkstra or A*.

The implementation uses dependency-free JavaScript. Costs, heuristics, and intermediate arithmetic must stay in the exact safe-integer range of the runtime; values outside the supported range produce errors. This is a bounded-integer implementation rather than an arbitrary-precision solver. Graphs are immutable after construction. An unreachable goal is represented by an absent path and infinite cost; exported JSON uses `null` and CSV uses `INF` for non-finite distances.

For A*, $g(v)$ is the cheapest discovered start-to-v cost and

$$f(v) = g(v) + h(v). \tag{1}$$

The heuristic is a deterministic function with $h(v) \geq 0$ and $h(t) = 0$ at the goal t. Optimality requires admissibility: $h(v) \leq d(v, t)$, with $d(v, t) = \infty$ if no goal is reachable from v. This lower-bound property is a mathematical obligation of the caller; checking that a supplied number is finite does not establish it. Consistency is the stronger edgewise condition

$$h(u) \leq w(u, v) + h(v). \tag{2}$$

Equal priorities prefer larger g, then insertion order. This choice matters for the reported expansion counts.

3 Search procedures and invariants

3.1 Breadth-first and depth-first search

BFS uses an array queue with a monotonically advancing head index. It marks vertices when inserted and stores predecessor links. Queue order processes distances in nondecreasing order: an undiscovered successor of a vertex at distance d has shortest distance $d + 1$. Each vertex enters the queue at most once. Parent links reconstruct a shortest path on a unit-cost graph.

DFS uses stack frames containing a vertex and its next adjacency index. This preserves recursive traversal order while avoiding language-level recursion limits. Every discovered vertex has a path from the start through its predecessors. A vertex is discovered only once; all its outgoing edges are eventually considered unless the goal has already been found. These invariants establish reachability on finite graphs, not shortest-path optimality.

Bidirectional BFS maintains forward layers from the start and reverse-edge layers from the goal. It expands the smaller frontier, always completing a layer and choosing its least-cost intersection. If the two previously discovered balls have radii a and b and are disjoint, no start-to-goal path has length at most $a + b$. After one radius grows by one, an intersection yields a path of length at most $a + b + 1$. The previous disjointness rules out a shorter answer. Reverse adjacency is essential for directed input.

3.2 Priority search with reopening

The implementation of A* maintains a binary min-heap, a best-cost array, and predecessor links. Its essential loop is:

1. Remove an entry with minimum f. Discard it if its recorded g is obsolete.
2. If the vertex is the goal, reconstruct and return its path.
3. For each outgoing edge, compute the candidate cost. If it strictly improves the successor's best cost, update the predecessor and insert a new heap entry.
4. Permit an improved vertex to be processed again even if it was processed earlier.

The last rule is required for admissible but inconsistent heuristics. For example, use edges

$$0 \to 1 : 3, \qquad 0 \to 2 : 1, \qquad 2 \to 1 : 1, \qquad 1 \to 3 : 3$$

and heuristic $h = (0, 0, 4, 0)$ for goal 3. Vertex 1 is initially processed with cost 3, then improved through vertex 2 to cost 2. Reopening produces the optimal route $0 \to 2 \to 1 \to 3$ of cost 5.

Theorem 1 (Finite-graph correctness under the stated contract)**.** *Suppose the graph is finite with nonnegative edge costs, the cached heuristic is deterministic, nonnegative, admissible, and zero at the goal, and every cost or priority sum evaluated during the run is an exactly represented safe integer. Then the implemented A^* terminates and returns an optimal path when the goal is reachable, or reports that no path exists otherwise.*

Proof. Every finite best-cost label is the cost of a path found by relaxation. Fix an optimal start-to-goal path of cost C^*. Its start is initially in the heap at its optimal prefix cost. When a vertex on this path is removed at its optimal prefix cost, relaxing its next edge inserts the next vertex at its optimal prefix cost, unless that next vertex was already removed at that cost. In the latter case, its outgoing edges have already been relaxed. Following the path therefore shows that, until the goal is removed at cost C^*, the heap contains a path vertex at its optimal prefix cost. Admissibility makes that entry's f at most C^*. A goal entry has $f = g$ because $h(t) = 0$, and its path cost cannot be below C^*. Therefore a goal removed from the minimum heap has cost exactly C^*; a more expensive goal cannot be removed while that no-greater-f entry remains.

Each successful relaxation strictly decreases a vertex's best label. Labels are nonnegative integers in the finite supported range, so each vertex can be improved only finitely many times. There are finitely many vertices and edges; hence there are finitely many successful relaxations, heap insertions, and removals. If the goal is reachable, successive relaxation along a path eventually discovers it. If the heap empties without reaching it, no reachable path to the goal exists. This termination argument alone is not a polynomial bound in the input length. □

With a consistent heuristic, vertices need not be reopened. Setting $h = 0$ gives Dijkstra's algorithm. A separate test intentionally uses an overestimating heuristic and demonstrates a suboptimal result, making the contract's limitation explicit.

4 Complexity accounting

Here E counts adjacency entries. Construction, validation, and reverse adjacency take $O(|V| + |E|)$ time and space. Each search initializes arrays of length $|V|$.

Method	Worst-case time	Auxiliary space
BFS / iterative DFS	$O(\|V\| + \|E\|)$	$O(\|V\|)$
Bidirectional BFS	$O(\|V\| + \|E\|)$	$O(\|V\|)$
Consistent A^* / Dijkstra	$O(\|V\| + (\|E\| + 1)\log(\|E\| + 2))$	$O(\|V\| + \|E\|)$

The heap bound includes duplicate entries retained until removal; it is not the bound for an indexed decrease-key heap. Constant-cost heuristic evaluation is assumed. Otherwise its total computation must be added. Inconsistent admissible heuristics preserve optimality with reopening but do not inherit the once-per-vertex expansion bound. These are RAM-model operation bounds for supported integers, not bit-complexity claims for unbounded numeric data.

The queue and frame choices improve implementation behavior relative to naive queue shifting, copying whole paths, or recursive traversal. They do not improve the general asymptotic bounds of BFS and DFS. Bidirectional search likewise has no universal worst-case asymptotic improvement over ordinary BFS.

5 Graph matrix collection

Five illustrative graphs are generated locally: a directed diamond, the weighted reopening example, a weighted disconnected undirected graph, a five-cycle, and a 4×4 grid. They are not a downloaded benchmark dataset. Each exports nine matrices in CSV and JSON, giving 45 CSV files plus graph descriptions and an index.

Binary adjacency A records edge existence. Edge multiplicity records parallel edges. W sums weights between each ordered pair, while a separate minimum-edge-cost matrix records the cheapest individual edge. This distinction matters: a summed weight is not the correct transition cost for shortest paths with parallel edges. Binary adjacency also distinguishes zero-cost edges from absence.

For incidence matrix B, an edge column has -1 at its source and $+1$ at its destination. Self-loop columns are zero. Define $D_{uu} = \sum_v W_{uv}$ and $L = D - W$. For undirected graphs, with C the diagonal matrix of input-edge weights,

$$L = BCB^{\mathsf{T}}, \qquad z^{\mathsf{T}}Lz = \sum_{e=(u,v)} w_e(z_v - z_u)^2 \geq 0. \tag{3}$$

Thus the undirected L is symmetric positive semidefinite. On directed graphs $D - W$ uses the out-degree convention and need not be symmetric; BCB^{T} instead describes the corresponding undirected edge collection. These objects must not be conflated.

For the undirected five-cycle the exported Laplacian is

$$L = \begin{pmatrix} 2 & -1 & 0 & 0 & -1 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ -1 & 0 & 0 & -1 & 2 \end{pmatrix}.$$

Reachability includes zero-length paths. Shortest-path matrices are computed independently by Floyd–Warshall and checked against the search implementation. Dense square matrices use $O(|V|^2)$ storage, dense incidence uses $O(|V||E|)$ storage, and Floyd–Warshall takes $O(|V|^3)$ time. For large sparse graphs, edge lists and query-specific distances are preferable to materializing this entire collection.

6 Validation and reproducible experiments

The recorded runtime is Node.js v24.22.1 on Windows. Fourteen automated tests passed. Exhaustive enumeration covers $2^{4 \cdot 3} = 4,096$ directed simple graphs and all 16 ordered start/goal pairs, or 65,536 queries per method. BFS, bidirectional BFS, and zero-heuristic A* are compared with independent Bellman–Ford relaxation. DFS reachability and returned paths are checked separately.

Another 100 seeded weighted graphs have eight vertices each. All ordered start/goal pairs yield 12,800 Dijkstra/A* comparisons. Admissible heuristic values are constructed from independent distances for testing; this is not proposed as a cost-free heuristic for a new problem. The seed is 20261030. Targeted cases cover inconsistent heuristics, stale entries, zero costs, cycles, invalid inputs, overflow, parallel edges, and a 30,000-vertex DFS chain.

Matrix tests compare every collected distance with the search results, check undirected BCB^{T} identities and symmetry, verify row sums of L, and confirm grid Manhattan distances. An export check parsed all CSV files and compared them with their JSON equivalents and regenerated data.

Both scenarios use unit-cost, undirected, four-neighbor grids. In the wall case, vertical walls occur at columns $5, 11, 17, 23, 29, 35$, with gaps alternating between rows 38 and 1 under

Scenario	Method	Path cost	Processed vertices
Open 40×40	BFS	78	1600
	DFS	1560	1561
	Bidirectional BFS	78	1560
	Dijkstra	78	1600
	A*, Manhattan	78	79
Alternating wall gaps	BFS	300	1366
	DFS	872	1361
	Bidirectional BFS	300	1359
	Dijkstra	300	1366
	A*, Manhattan	300	1239

Table 1: Deterministic operation counts from `demo.mjs`. The processed-vertex metric counts the goal even though its outgoing edges are not scanned. Construction is excluded.

zero-based indexing. Manhattan distance remains a lower bound and is consistent. Equal-priority depth preference explains the particularly small open-grid count. Obstacles weaken this advantage. DFS's longer paths illustrate that reachability and shortest-path search are different objectives.

Counts in Table 1 are not wall-clock speedup measurements. Heap overhead, heuristic computation, construction, and retained entries influence actual runtime and memory. A reported peak frontier is algorithm-specific: DFS measures active frames, BFS pending entries, A* heap entries including stale entries, and bidirectional BFS retained current/next layers. The values are not interchangeable memory measurements.

7 What the graph results imply for SAT

Proposition 1 (Depth-only heuristic)**.** *In a SAT assignment tree with unit-cost edges, fixed variable order, and goals restricted to satisfying complete assignments of n variables, the heuristic $h = n - d$ at assignment depth d gives the same priority to every generated state.*

Proof. The path cost is $g = d$, so $f = d + (n - d) = n$. The heuristic is admissible: every reachable goal requires exactly $n - d$ further assignments, while an unreachable goal has infinite remaining distance. Thus the heuristic supplies no priority-based distinction between promising and unpromising assignments. A complete binary tree through depth n has $2^{n+1} - 1$ vertices, but this count is not a lower bound for every solver or a claim that A* visits every node. A polynomial guarantee for a proposed search method needs a separate bound on the states and work it actually generates, for example through sound pruning, merging, or a more compact representation. □

If partial satisfying assignments are accepted as goals, $n-d$ may overestimate their remaining distance and is not automatically admissible. A goal predicate and heuristic must be specified together.

Proposition 2 (Exact residual-function merging)**.** *For the family $F_k = \bigwedge_{i=1}^{k}(x_i \vee y_i)$, assigning all x variables first produces 2^k distinct residual Boolean functions over the y variables.*

Proof. For the set S of indices with $x_i = 0$, the residual is $R_S = \bigwedge_{i \in S} y_i$. If $S \neq T$, exchange their names if necessary and choose $j \in S \setminus T$. Set $y_j = 0$ and all other y variables to 1. Then $R_S = 0$ and $R_T = 1$, so the functions differ. There are 2^k subsets. □

Consequently, exact residual-function equality alone does not ensure polynomially many states in this order. The encoded formula length is $O(k \log k)$ with binary variable identifiers,

and 2^k is superpolynomial in that length. This is nevertheless an easy satisfiable family: setting every y_i to 1 suffices. The proposition concerns one representation and merging criterion, not a lower bound for all SAT algorithms or a necessity to enumerate every state.

A polynomial-size clause-variable graph also does not make ordinary reachability a SAT decider: reachability does not enforce one globally consistent assignment satisfying every clause. Conversely, an assignment graph may represent the choices faithfully while being exponentially large. A general polynomial-time claim needs both a correct encoding and a polynomial bound on construction, state generation, merging, heuristic computation, arithmetic, and termination on unsatisfiable inputs.

8 Reproduction, limitations, and next milestone

The repository is `https://github.com/n0711/P_EQUALS_NP`. From its root, run:

```
node --test
node demo.mjs
node collect-matrices.mjs
```

The paper is built from the `paper` directory by running `xelatex graph-search.tex` twice. The local documentation records matrix conventions, input contracts, and validation scope.

Finite experiments identify implementation errors and measure chosen examples; they cannot establish a universal polynomial bound. The current code requires explicit finite graphs and supported integer arithmetic. No SAT solver is implemented here. A useful next milestone is a representation and pruning rule checked on a tractable restricted family, such as 2-SAT, followed by a precise lemma describing the generalization to unrestricted CNF. The substantive outcome of this report is a reproducible baseline and explicit proof obligations.

Licence and preparation

Copyright © 2026 Charalampos Nadiotis. This manuscript and its XeLaTeX source are distributed under the ROS3-derived *Civilian-Use Licence (No Military Use), Version 1.0*, reproduced in Appendix A. The restriction applies to the licensed manuscript, including its adaptations. Separately licensed works and independent implementations retain their own legal status. The report and implementation were prepared with AI-assisted drafting and coding; this disclosure is not a claim of independent peer review.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968. doi:10.1109/TSSC.1968.300136.

[2] C. Nadiotis, *On the Proposition P = NP: Formal Foundations, Necessary Proof Obligations, and Research Directions*, PELAOS Tech, 30 October 2026. ResearchGate manuscript.

A Civilian-Use Licence (No Military Use), Version 1.0

Copyright 2026 Charalampos Nadiotis.

1. Licensed material and scope. These terms apply to the original copyrightable content of this manuscript, *Deterministic Graph Search and Matrix-Based Validation: A Reproducible Framework for SAT-Oriented Research*, technical report version 1.0, 30 October 2026, including its XeLaTeX source and compiled PDF. Referenced third-party works and separately licensed software are excluded. Only rights held by the copyright holder are granted. No patent or trademark rights are granted.

2. Permitted civilian reuse. Subject to these terms, the copyright holder grants worldwide, royalty-free, non-exclusive permission to reproduce, distribute, communicate, translate, and adapt the licensed material for civilian purposes, including commercial civilian activities.

3. Attribution and redistribution. Retain the author's name, manuscript title, copyright notice, and this licence. Identify changes and do not imply endorsement by the author. Copies and adaptations containing the licensed material must include these terms and retain the prohibition on military use. Recipients receive permission under these same terms; a distributor may not grant military-use permission for the licensed material.

4. Military use prohibited. The licensed material and adaptations may not be used for military purposes. A military purpose is an activity intended to support armed forces, military operations, or military capabilities. This includes research, design, development, integration, training, simulation, testing, procurement, deployment, operation, or maintenance of military weapons, targeting systems, unmanned systems, command-and-control systems, or military intelligence, surveillance, and reconnaissance systems. The prohibition applies both to direct military use and to work by civilian organisations or individuals intended for a military end use.

5. Termination. Permission terminates automatically for a user who breaches these terms. That user must stop exercising the permissions granted by this licence. Reinstatement requires written permission from the copyright holder.

6. Exceptions and warranty. Nothing in these terms restricts a use that applicable law permits without copyright permission. The material is provided *as is*, without warranties, to the extent permitted by law. These terms regulate copyright reuse of the manuscript; they do not restrict independent implementations or change rights already granted for separately licensed software.

Provenance and software boundary. These terms adapt the author's ROS3-Limassol manuscript licence dated 25 October 2026. The work title, edition, date, and attribution clause have been updated for this report; the ROS3-specific initial-project credit is not transferred to this different work. No new software licence is granted by this manuscript licence.